🎯 1. Start With a Real-Life Scenario (VERY IMPORTANT)

👉 "Imagine you are working in a kitchen 👨‍🍳
You have 4 workers (CPUs), and multiple dishes (processes).

Now:

- If all dishes go to **only 1 worker**, others are idle 😴
- If you **assign specific dishes to specific workers**, work becomes faster ⚡

👉 This is exactly what we do in Operating Systems using **CPU Affinity**."

🧠 2. Concept Building

🔹 Step 1: What is CPU / Core?

Explain:

- CPU has multiple **cores**
- OS treats each core as a **separate CPU**

👉 Example:

- 1 processor with 4 cores = 4 logical CPUs

🔹 Step 2: What is Multiprocessing?

👉 Simple line:
"Running multiple processes at the same time using multiple CPUs"

🔹 Step 3: What is CPU Affinity?

👉 Simple definition:
"Binding a process to a specific CPU"

👉 Example:

- Process A → CPU 0
- Process B → CPU 1

🔹 Step 4: Why do we use it?

- Better performance
- Avoid CPU switching overhead

- Control execution

💻 **3. Ubuntu Commands (VERY IMPORTANT PART)**

✅ **Check CPU Information**

lscpu

👉 Explain:

- Total CPUs
- Cores
- Threads

✅ **Check number of CPUs**

nproc

✅ **See running processes**

top

or

htop

✅ **Check process CPU**

ps -o pid,psr,comm

👉 psr = which CPU is running the process

🔥 **4. Taskset Command (Main Tool)**

🔹 **Check affinity of a process**

taskset -p <PID>

🔹 **Set affinity**

taskset -p 1 <PID>

👉 Means run only on CPU 0

🔹 **Run program on specific CPU**

taskset 2 ./a.out

👉 Runs on CPU 1

🔹 **CPU Mask Concept (VERY IMPORTANT)**

Explain like this:

Mask	Binary	CPU
1	0001	CPU 0
2	0010	CPU 1
3	0011	CPU 0 & 1

💻 **5. First Simple C Program (Processors Count)**

✅ **Code**

```
#include <sys/sysinfo.h>
#include <stdio.h>

int main() {
   printf("Configured CPUs: %d\n", get_nprocs_conf());
   printf("Available CPUs: %d\n", get_nprocs());
   return 0;
}
```

🔍 **Line-by-line Explanation**

- get_nprocs_conf() → total CPUs
- get_nprocs() → active CPUs

💻 **6. Check Current CPU of Process**

✅ **Code**

```
#include <sched.h>
#include <stdio.h>
```

```
int main() {
   printf("Running on CPU: %d\n", sched_getcpu());
}
```

🔍 **Explanation**

- sched_getcpu() → tells current CPU

💻 7. Get Affinity Program

✅ **Code**

```
#define _GNU_SOURCE
#include <sched.h>
#include <stdio.h>
#include <sys/sysinfo.h>

int main() {
   cpu_set_t set;
   CPU_ZERO(&set);

   sched_getaffinity(0, sizeof(set), &set);

   int n = get_nprocs_conf();

   for(int i=0; i<n; i++) {
      printf("CPU %d: %d\n", i, CPU_ISSET(i, &set));
   }

   return 0;
}
```

🔍 **Explanation**

- CPU_ZERO → clear mask
- sched_getaffinity → get CPUs

- CPU_ISSET → check CPU active or not

💻 8. Set Affinity Program

✅ Code

```
#define _GNU_SOURCE
#include <sched.h>
#include <stdio.h>

int main() {
   cpu_set_t set;

   CPU_ZERO(&set);
   CPU_SET(1, &set);  // Set CPU 1

   sched_setaffinity(0, sizeof(set), &set);

   printf("Affinity set to CPU 1\n");
}
```

🔍 Explanation

- CPU_SET(1) → bind to CPU 1
- sched_setaffinity → apply setting

🎯 ✅ Question 1 (CPU Selection + Affinity)

❓ Question:

👉 Write a C program that:

- System ke available CPUs display kare
- User se CPU number input le
- Current process ko us CPU par bind kare
- Aur phir show kare ke process ab kis CPU par run ho raha hai

💻 **Code:**

```
#define _GNU_SOURCE
#include <sched.h>
#include <stdio.h>
#include <sys/sysinfo.h>

int main() {
   int cpu;
   int n = get_nprocs_conf();

   printf("Total CPUs: %d\n", n);

   // Show CPUs
   for(int i = 0; i < n; i++) {
      printf("CPU %d available\n", i);
   }

   // User input
   printf("Enter CPU number: ");
   scanf("%d", &cpu);

   // Create mask
   cpu_set_t set;
   CPU_ZERO(&set);
   CPU_SET(cpu, &set);

   // Set affinity
   sched_setaffinity(0, sizeof(set), &set);

   // Show current CPU
   printf("Now running on CPU: %d\n", sched_getcpu());
```

```
    return 0;
}
```

✅ Question 1

Write a C program that:

- Displays the total number of CPUs available in the system
- Takes a CPU number as input from the user
- Sets the current process affinity to that CPU
- Displays the CPU on which the process is running after setting affinity

💻 Code:

```
#define _GNU_SOURCE
#include <sched.h>
#include <stdio.h>
#include <sys/sysinfo.h>

int main() {
    int cpu;
    int n = get_nprocs_conf();

    printf("Total CPUs: %d\n", n);

    for(int i = 0; i < n; i++) {
        printf("CPU %d available\n", i);
    }

    printf("Enter CPU number: ");
    scanf("%d", &cpu);

    cpu_set_t set;
    CPU_ZERO(&set);
    CPU_SET(cpu, &set);

    sched_setaffinity(0, sizeof(set), &set);

    printf("Now running on CPU: %d\n", sched_getcpu());

    return 0;
}
```

✅ Question 2

Write a C program that:

- Creates three child processes using `fork()`
- Each process performs a different task:
  - Process 1: Calculate square
  - Process 2: Calculate cube
  - Process 3: Double the number
- Each process must run on a different CPU using CPU affinity

💻 Code:

```
#define _GNU_SOURCE
#include <sched.h>
#include <stdio.h>
#include <unistd.h>

void set_cpu(int cpu) {
    cpu_set_t set;
    CPU_ZERO(&set);
    CPU_SET(cpu, &set);
    sched_setaffinity(0, sizeof(set), &set);
}

int main() {
    int num = 5;

    if(fork() == 0) {
        set_cpu(0);
        printf("Square: %d (CPU %d)\n", num*num, sched_getcpu());
    }

    else if(fork() == 0) {
        set_cpu(1);
        printf("Cube: %d (CPU %d)\n", num*num*num, sched_getcpu());
    }

    else if(fork() == 0) {
        set_cpu(2);
        printf("Double: %d (CPU %d)\n", num*2, sched_getcpu());
    }

    return 0;
}
```